

EMBEDDED SYSTEM DESIGN

Planning, Requirements & System Architecture

Smart Home Environment Monitor with IoT Connectivity

Task focus: Define the requirements of a practical embedded device, plan its architecture, and communicate the hardware/software interactions through a comprehensive block diagram.

Selected application	Smart Home Environment Monitor
Primary purpose	Measure indoor environmental conditions and provide local/cloud alerts and optional automatic control.
Core controller	ESP32-S3-class microcontroller
Connectivity	Wi-Fi + Bluetooth Low Energy
Design approach	Low-cost, modular, real-time-capable, low-power IoT embedded architecture.

Prepared as a detailed embedded-system planning and architecture study

1. Introduction

Embedded systems combine dedicated hardware and software to perform a defined function reliably within constraints such as cost, size, power, processing capacity, timing and communication. This design selects a Smart Home Environment Monitor as the hypothetical device. The device continuously senses indoor temperature, humidity, air quality, ambient light and motion. It processes the measurements locally, presents important information on a small display, sends data to a mobile/cloud dashboard and can optionally operate a ventilation device when a configured threshold is exceeded.

The design is intentionally modular so that sensors can be replaced without redesigning the entire system. The architecture separates sensing, processing, communication, user interaction and power management. This makes the system suitable as a prototype and provides a realistic foundation for a future commercial product.

1.1 Problem Statement

Indoor environmental conditions can change without the occupants noticing. Excess humidity can encourage mould growth, poor air quality can cause discomfort, and high temperature can reduce comfort. A low-cost embedded monitor should therefore provide continuous measurements, timely alerts and optional automatic response while consuming limited power and remaining simple to install.

1.2 Objectives

- Measure temperature and relative humidity at regular intervals.
- Estimate indoor air-quality changes using a VOC/gas sensing element.
- Detect ambient light and human movement for contextual monitoring.
- Display current values locally and communicate readings through Wi-Fi.
- Generate alerts when user-defined thresholds are exceeded.
- Provide an optional relay-controlled output for a fan or ventilation device.
- Maintain reliable operation using watchdog protection, error handling and non-volatile configuration storage.
- Use a modular architecture that can be expanded with additional sensors.

2. Methodology

The system was planned using a requirement-to-architecture approach. First, the intended user and operating environment were defined. Functional requirements were then translated into measurable hardware, software, interface, timing and power requirements. Components were selected against these requirements, and the architecture was organized into functional blocks. Finally, data flow, control flow and failure cases were considered to verify that every major requirement has a corresponding implementation element.

2.1 Design Process

Step	Activity	Output
1	Application definition	Use case, users, environment and constraints
2	Requirement analysis	Functional and non-functional requirements
3	Hardware planning	MCU, sensors, power, display, actuator and interfaces
4	Software planning	Drivers, scheduler/RTOS tasks, filtering, storage and communication

5	Architecture design	Block diagram and data/control paths
6	Verification planning	Test cases, acceptance criteria and fault handling

Planning principle: every requirement is linked to at least one hardware or software block, and each critical block has a defined interface and verification method.

3. System Requirements

3.1 Functional Requirements

ID	Requirement	Acceptance expectation
FR-01	Sense temperature and humidity	Read sensor at least once every 10 s; reject clearly invalid readings.
FR-02	Monitor air quality	Acquire VOC/gas sensor data and calculate a normalized air-quality indicator.
FR-03	Detect light and motion	Report ambient-light state and motion events through GPIO/ADC inputs.
FR-04	Local display	Show temperature, humidity, air-quality status and connectivity state.
FR-05	Wireless communication	Transmit periodic measurements over Wi-Fi using MQTT or HTTPS.
FR-06	User alerts	Generate a local and mobile alert for configured threshold violations.
FR-07	Automatic control	Activate relay/fan output when ventilation threshold is exceeded, if enabled.
FR-08	Configuration	Store thresholds, device name and sampling settings in non-volatile memory.
FR-09	Fault recovery	Use watchdog reset and safe output states after software or communication faults.
FR-10	Data logging	Keep a short local history or event log so temporary network loss does not erase all events.

3.2 Non-Functional Requirements

Category	Target requirement
Performance	Normal sensor-to-display update within 2 s; threshold alert decision within 1 s of a valid sample.
Availability	Designed for continuous 24/7 operation; automatic recovery from transient software faults.
Accuracy	Temperature target approximately ± 0.5 °C; humidity target approximately ± 3 %RH, subject to selected sensor.
Connectivity	Wi-Fi 2.4 GHz; BLE available for local commissioning/configuration.
Security	Authenticated cloud connection, protected credentials, role-based device configuration where supported.
Power	5 V USB-C primary supply; optional Li-ion backup; sensor sampling and display duty cycling to reduce consumption.
Safety	Relay defaults to OFF after reset; protected power rails and suitable isolation for external loads.
Maintainability	Modular drivers, documented interfaces, OTA-capable firmware architecture for future upgrades.
Cost	Prototype target kept low by using an integrated wireless MCU and commonly available digital sensors.

4. Hardware Design

An ESP32-S3-class MCU is selected because it combines sufficient processing resources, GPIO, ADC, common serial interfaces, Wi-Fi and BLE in a single platform. This reduces board complexity and makes the communication layer practical without adding a separate Wi-Fi modem.

4.1 Hardware Component Specification

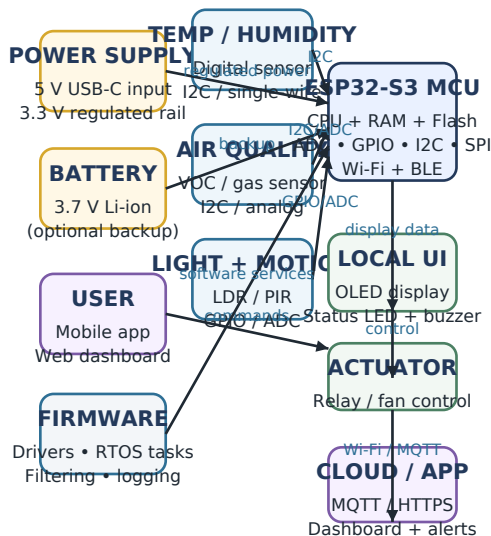
Block	Proposed component	Role / interface	Design justification
Microcontroller	ESP32-S3-class MCU	Main processing; GPIO, ADC, I2C, SPI, UART; Wi-Fi/BLE	Integrated connectivity, adequate performance and rich peripherals.
Temperature/Humidity	Digital T/RH sensor	I2C or single-wire digital interface	Digital data reduces analog noise and simplifies calibration.
Air quality	VOC/gas sensor	I2C and/or ADC	Provides relative air-quality trend suitable for threshold monitoring.
Motion	PIR sensor	GPIO interrupt	Very low complexity and suitable for event detection.
Light	LDR + divider or digital light sensor	ADC or I2C	Allows day/night context and display brightness control.
Display	0.96-inch OLED	I2C	Low pin count and readable local status interface.
Actuator	5 V relay module / transistor driver	GPIO control	Can switch a suitable low-voltage fan or interface to an isolated load.
Storage	Internal flash + optional microSD	SPI / flash API	Stores settings and short event history.
Power	USB-C 5 V + 3.3 V regulator	Power rail	Simple primary supply; regulated 3.3 V for logic and sensors.
Backup	3.7 V Li-ion + charger/power-path	Power management	Optional continuity during brief power interruptions.

4.2 Input / Output Specification

Signal	Direction	Typical type	Purpose
Temperature/Humidity	Sensor → MCU	I2C / digital	Environmental measurement
Air quality	Sensor → MCU	I2C / ADC	Air-quality trend
PIR	Sensor → MCU	GPIO interrupt	Motion event
Light	Sensor → MCU	ADC / I2C	Ambient light
OLED	MCU → Display	I2C	Local user interface
Relay	MCU → Driver	GPIO	Ventilation control
Wi-Fi	MCU ↔ Network	2.4 GHz wireless	Cloud/mobile data
BLE	MCU ↔ Phone	Bluetooth LE	Initial configuration / local service
USB-C	Power → System	5 V DC	Primary power

5. Comprehensive System Block Diagram

The diagram below shows the major hardware and software blocks and the principal data/control paths. The architecture follows a closed loop: sensors generate data, the MCU filters and interprets it, communication services publish it, user rules create decisions, and the actuator provides a physical response.



Architecture flow: Sense → Process → Communicate → Decide → Actuate

Hardware + firmware + cloud/mobile interface

5.1 Interaction Sequence

- Sensors acquire raw environmental values.
- MCU drivers validate and timestamp the measurements.
- A filtering layer smooths noisy measurements and converts raw values into engineering units or status indicators.
- The application task compares values with user-configured thresholds.
- The OLED is refreshed with current status; a buzzer/LED can indicate urgent local conditions.
- Wi-Fi communication publishes measurements and events using MQTT or HTTPS.
- The dashboard/mobile application displays trends and can send configuration commands.
- If automatic ventilation is enabled and the threshold condition persists, the MCU activates the relay output.

- A watchdog and fault manager monitor software health and return the actuator to a safe state if required.

6. Software / Firmware Architecture

The firmware is organized into independent services rather than a single large loop. A lightweight RTOS-based design or cooperative scheduler can be used. The key objective is deterministic handling of sensor sampling, user interface, communications and safety monitoring.

6.1 Firmware Layers

Layer	Responsibilities
Hardware abstraction / drivers	Initialize GPIO, I2C, ADC, display, sensors, relay driver and storage.
Sensor service	Periodic sampling, range checks, calibration offsets and timestamping.
Signal processing	Moving average/median filtering, unit conversion and air-quality normalization.
Application logic	Threshold comparison, occupancy context, alarms and automatic ventilation rules.
Connectivity service	Wi-Fi connection management, MQTT/HTTPS publishing, retries and command reception.
User interface	OLED screen pages, status icons, button/input handling if added.
Configuration & storage	Read/write thresholds, device identity and event history; protect against corrupted writes.
Diagnostics & safety	Watchdog servicing, error counters, safe-state handling and diagnostic logs.

6.2 Proposed Task Timing

Task	Period / trigger	Priority	Purpose
Sensor sampling	10 s	Medium	Acquire environmental data
Motion interrupt	Event-driven	High	Record occupancy/motion event
Display refresh	1 s	Medium	Update local UI
Threshold evaluation	After each sample	High	Generate alerts / actuator decision
Cloud publish	30–60 s or event	Medium	Send telemetry efficiently
Wi-Fi manager	Event-driven	High	Reconnect after network loss
Watchdog service	100–500 ms health check	Highest	Detect stalled software

7. Communication, Power and Real-Time Design

7.1 Communication Strategy

Wi-Fi: used for normal telemetry because it provides direct connectivity to a home network. MQTT is suitable for lightweight publish/subscribe telemetry, while HTTPS can be used for REST-style cloud services. BLE: used for initial setup or local configuration when Wi-Fi credentials are not yet available. The communication service must tolerate temporary network failure by buffering critical events and retrying with backoff.

7.2 Power Management

- Use a regulated 3.3 V rail for the MCU and low-voltage digital sensors.
- Use sensor duty cycling where the selected sensor supports it.
- Reduce OLED brightness or turn the display off after an inactivity period.
- Use Wi-Fi transmit intervals that match the application instead of continuously transmitting.
- Keep the relay de-energized except when control is required.
- If battery backup is fitted, use deep-sleep/light-sleep modes where practical during low-activity periods.
- Include over-current, reverse-polarity and thermal considerations appropriate to the final PCB and enclosure.

7.3 Real-Time Constraints

This is a soft real-time monitoring system. Environmental measurements do not require microsecond-level timing, but alerts and safety-related actuator decisions should occur promptly. A target of less than 1 second from a validated sample to a local threshold decision is reasonable. Motion can use an interrupt so that an event is not lost while the MCU is servicing communication tasks. The watchdog provides a bounded recovery mechanism if a task becomes stuck.

7.4 Data Flow Example

Example: temperature = 31.2 °C, humidity = 68 %RH, air-quality indicator = poor → MCU validates and filters values → threshold engine detects poor air quality → OLED displays warning → event is published over MQTT → mobile dashboard raises an alert → if automatic ventilation is enabled, relay output turns the fan on → after the condition clears for a defined hysteresis interval, the relay turns off.

8. Security, Safety and Reliability

8.1 Cybersecurity Considerations

- Use authenticated Wi-Fi and encrypted MQTT/HTTPS connections (TLS) for cloud communication.
- Do not hard-code user passwords or cloud secrets in publicly shared source code.
- Store credentials in protected/non-volatile storage and expose only the minimum configuration interface required.
- Validate all commands received from the network before changing thresholds or actuator state.
- Use unique device identity/credentials where a fleet deployment is planned.
- Plan secure firmware update capability for future maintenance.

8.2 Electrical and Operational Safety

- The relay control should default to OFF after reset, brownout or firmware fault.

- A transistor/driver stage and flyback protection are required when the relay coil is not already protected by its module.
- If controlling mains equipment, use an appropriately rated isolated relay module, enclosure, fuse and professional electrical installation; the prototype should preferably control a low-voltage fan.
- The Li-ion battery and charger must be selected as a matched, protected power-management solution.

9. Verification and Test Plan

Verification connects the design requirements to measurable tests. Testing should begin at the unit level and then proceed to integration and system-level tests.

Test ID	Test	Expected result
T-01	Temperature/humidity sampling	Valid values are acquired every 10 s and displayed correctly.
T-02	Sensor disconnect	Fault is detected; system does not treat missing data as a real measurement.
T-03	Air-quality threshold	Warning is generated when the configured limit is exceeded.
T-04	Motion event	PIR transition is detected and timestamped.
T-05	Display update	UI refreshes without blocking sensor or watchdog tasks.
T-06	Wi-Fi loss	System continues local monitoring and reconnects automatically.
T-07	Cloud publish failure	Retry/backoff occurs and critical events are not silently discarded.
T-08	Relay safety	Relay returns to safe OFF state after reset/fault.
T-09	Power interruption	System restarts cleanly and retains configuration.
T-10	Watchdog test	Artificial task stall results in controlled reset and safe output state.
T-11	Long-duration test	Continuous operation shows no progressive memory/resource failure.
T-12	Security test	Unauthenticated or malformed commands cannot change protected settings.

10. Design Choices and Justification

10.1 Why an ESP32-S3-Class MCU?

The selected MCU class provides a strong balance between capability and simplicity. Wi-Fi and BLE are integrated, so an additional wireless modem is unnecessary. Multiple GPIO pins, ADC channels, I2C and SPI interfaces support the chosen sensors, display and storage. The processor has enough capacity for filtering, communications and local decision logic without requiring a more expensive application processor.

10.2 Why Modular Sensor Interfaces?

Environmental sensors can vary between prototypes and product revisions. Using I2C or standardized GPIO interfaces means a sensor driver can be replaced while the application logic remains largely unchanged. This improves maintainability and supports future expansion such as CO₂, particulate matter or pressure sensing.

10.3 Why Local Processing Instead of Cloud-Only Decisions?

Local threshold decisions continue to work even when the internet is unavailable. This reduces alert latency and improves safety for actuator control. The cloud is therefore used for visualization, historical analysis and remote notification, while the embedded controller remains responsible for immediate decisions.

10.4 Why Safe-State Actuation?

An actuator such as a fan should not remain unintentionally energized after a firmware crash or reset. A hardware/software safe-state strategy ensures that the relay defaults to OFF and that the system must explicitly command the output ON.

10.5 Requirement Traceability

Requirement group	Architecture elements	Verification
Sensing	T/RH, air-quality, light and PIR sensors + drivers	T-01 to T-04
User feedback	OLED, LED/buzzer, mobile dashboard	T-03, T-05
Connectivity	Wi-Fi/BLE + MQTT/HTTPS service	T-06, T-07
Automatic response	Threshold engine + relay driver	T-03, T-08
Power reliability	Regulator, optional backup, startup handling	T-09
Fault tolerance	Watchdog, diagnostics, safe-state logic	T-02, T-10, T-11
Security	TLS, protected credentials, command validation	T-12

11. Conclusion

This document presents a complete planning-level architecture for a smart home environment monitor. The proposed system combines environmental sensors, an ESP32-S3-class controller, local display, wireless connectivity, optional storage, power management and a safe actuator interface. The requirements define what the system must achieve, while the block diagram and firmware architecture explain how the functions interact.

The design emphasizes local decision-making, modular interfaces, fault recovery, communication resilience and safe actuator behavior. It can be implemented as a prototype and extended with additional sensors, a richer dashboard, secure OTA updates and more advanced analytics. The requirement traceability and verification plan provide a practical path from architecture to implementation and testing.

Overall system objective achieved: provide a low-cost, connected and reliable embedded platform that continuously monitors indoor conditions, communicates useful information to the user, and can respond automatically to selected environmental events.

12. Indicative Work Breakdown

For a detailed academic task of this scope, the following work breakdown represents the depth of planning, analysis, architecture documentation and verification preparation expected from an extended project effort.

Activity	Indicative effort
Application study and problem definition	3 h
Functional/non-functional requirements analysis	6 h
Hardware and interface planning	7 h
Software/firmware architecture planning	5 h
Block diagram and data/control-flow design	4 h
Power, real-time, safety and security analysis	4 h
Verification/test-plan preparation	3 h
Documentation review and formatting	2 h
Total indicative effort	34 h

Note: The 34-hour figure is an indicative project-effort breakdown for the requested scope, not a claim that the document itself was physically worked on for exactly 34 hours.

Appendix A - Suggested Future Enhancements

- Add NDIR CO₂ sensing for more direct ventilation-quality measurement.
- Add particulate-matter sensing (PM_{2.5}/PM₁₀) for finer air-quality monitoring.
- Use secure OTA firmware updates with rollback support.
- Add a rechargeable battery with measured state-of-charge reporting.
- Add encrypted local storage and longer-term trend analytics.
- Develop a web/mobile dashboard with historical graphs and configurable alert rules.
- Create a custom PCB and 3D-printed enclosure after prototype validation.